

AI Study Assistant

Project Report

Project Type: Full-Stack Web Application

Frontend: React + Vite

Backend: Node.js + Express + MongoDB

AI Integration: OpenRouter API

1. Title Page

Title: AI Study Assistant

Purpose: A study management platform that helps users organize subjects and notes, then interact with an AI tutor for explanations and question answering.

2. Abstract

The AI Study Assistant project is a web-based academic support system designed to help students create subjects, manage study notes, and ask questions to an AI tutor. The application combines user authentication, subject-based note organization, and AI-assisted learning in one interface. The system uses a React frontend for user interaction, an Express and Node.js backend for API handling, MongoDB for persistent storage, and OpenRouter for AI responses.

The project demonstrates how modern full-stack technologies can be integrated to improve personal learning workflows.

3. Introduction

Students often keep study material in scattered places such as notebooks, chat messages, or random text files. This makes revision difficult and reduces productivity. The AI Study Assistant addresses this by giving users a dedicated platform where they can store notes subject-wise and directly interact with an AI tutor based on those notes. The system is meant to simplify note organization and make revision more interactive and personalized.

4. Problem Statement

Many students face difficulty in organizing study material and quickly understanding saved content during revision. Traditional note-taking systems do not provide intelligent support. The problem this project solves is the lack of a single system that can both manage notes and help explain them interactively through AI.

5. Literature Review

Existing study tools usually focus on either note storage or AI chat, but not both together in a lightweight personal workflow. Note-taking apps provide organization, while AI chat tools provide explanation. However, students still need to manually move information between systems. This project combines subject management, note storage, and tutoring into one workflow, reducing friction and improving usability.

6. Methodology

- Requirements were identified around subject creation, note management, authentication, and AI question answering.
- A MERN-style architecture was selected for ease of development and clear separation between frontend and backend.
- The backend was built first with REST routes for authentication, subjects, notes, and chat.
- The frontend was then developed to consume these APIs and provide a responsive user interface.
- AI integration was added to the chat flow so users can ask questions related to saved notes.
- UI refinements and layout fixes were applied to improve desktop and mobile usability.

7. Tools & Technologies

- Frontend: React, Vite, React Router, Axios, React Icons, Framer Motion
- Backend: Node.js, Express.js, Axios
- Database: MongoDB with Mongoose
- Authentication: JWT, bcryptjs
- AI Provider: OpenRouter API
- Versioning and Development: npm, nodemon

8. System Design

The system uses a client-server architecture. The React frontend handles routing and user interaction. The backend exposes REST APIs for registering users, logging in, creating subjects, adding notes, retrieving notes by subject, deleting notes, and chatting with AI. MongoDB stores users, subjects, and notes, with each subject and note tied to a specific authenticated user. The AI chat route fetches a selected note and sends the note content plus the student's question to the OpenRouter API.

9. Implementation

- User registration and login were implemented with password hashing and JWT-based authentication.
- Subject management allows users to create and fetch subject records linked to their account.
- Note management supports note creation, listing by subject, and secure deletion.
- Chat with AI was implemented through a protected route that reads note content and generates tutor-style responses.
- The frontend includes separate pages for authentication, dashboard, notes, and AI chat.
- Responsive styling and centered app-shell layout were added for improved desktop and mobile presentation.

10. Results & Analysis

The final system successfully supports the full basic study workflow: user authentication, subject creation, note creation, note viewing, note deletion, and AI-assisted question answering. The project also demonstrates stable integration between frontend and backend, plus a working connection to an external AI provider. From a usability perspective, the

application is significantly more practical after layout improvements and better tutor-style AI prompting.

11. Challenges & Limitations

- Initial issues were found in note saving and model import consistency in the backend.
- Several screens behaved like narrow mobile layouts on desktop and required shared layout corrections.
- AI integration required careful endpoint configuration and better prompt design to avoid repetitive answers.
- The current AI quality depends on external API availability and provider limits.
- There is no advanced note editing, tagging, or file upload support yet.

12. Future Scope

- Add note editing and richer markdown support.
- Generate quizzes, summaries, and flashcards directly from saved notes.
- Add suggested AI questions in the chat interface.
- Support attachments such as PDFs or images for study material.
- Improve analytics, revision tracking, and personalized study recommendations.
- Deploy the application with production-grade environment configuration.

13. Conclusion

The AI Study Assistant is a practical full-stack learning application that combines note organization with AI-powered tutoring. It addresses a real student need by placing subjects, notes, and study help in one system. The project demonstrates how frontend development, backend API design, database modeling, authentication, and AI integration can work together to deliver a useful academic tool.

14. References

- [React Documentation](#)
- [Vite Documentation](#)
- [Express.js Documentation](#)
- [MongoDB and Mongoose Documentation](#)
- [JWT Documentation](#)

- [OpenRouter API Documentation](#)

15. Appendix

Important Modules: Authentication, Subject Management, Note Management, AI Chat

Core Backend Files: backend/server.js, backend/controllers/authController.js, backend/controllers/subjectController.js, backend/controllers/noteController.js, backend/controllers/chatController.js, backend/services/aiService.js

Core Frontend Pages: frontend/src/pages/Login.jsx, Register.jsx, Dashboard.jsx, NotesPage.jsx, ChatPage.jsx

This report reflects the implemented project structure available in the current workspace.